

Procesamiento de un Archivo de Texto Grande mediante Descomposición Funcional

Explicación Detallada de las Soluciones y Conclusiones

Ejercicio – Descomposición Funcional (Pipeline) vs. Procesamiento Secuencial

1 Introducción

El presente informe documenta el desarrollo del ejercicio cuyo objetivo es aplicar la técnica de **descomposición funcional** para procesar un archivo de texto grande, comparando su desempeño frente a una solución secuencial. Se presenta el planteamiento del problema, la solución secuencial de referencia, la solución con descomposición funcional (pipeline), el análisis comparativo entre ambas, y las conclusiones obtenidas a partir de los resultados medidos.

1.1 Planteamiento del problema

Se dispone de un archivo de texto de gran tamaño (varios millones de líneas). Se requiere leer el archivo, dividirlo en líneas, limpiar cada línea (eliminar espacios en blanco al principio y al final), convertirla a mayúsculas, y escribir el resultado en un nuevo archivo. El reto es determinar si conviene descomponer este proceso en tareas independientes (pipeline) y qué efecto tiene esa descomposición sobre el rendimiento frente a una solución secuencial.

2 Solución 1: Procesamiento Secuencial

2.1 Descripción

La solución secuencial realiza todas las transformaciones en un único recorrido del archivo, línea por línea, sin funciones intermedias ni estructuras de datos adicionales.

Listing 1: Procesamiento secuencial línea por línea

```
1 def procesar_texto_secuencial(ruta_entrada, ruta_salida):
2     """Procesa el archivo de texto secuencialmente."""
3     try:
4         with open(ruta_entrada, 'r') as f_in, open(ruta_salida, 'w') as f_out:
5             for linea in f_in:
6                 linea_limpia = linea.strip()
7                 linea_mayusculas = linea_limpia.upper()
8                 f_out.write(linea_mayusculas + '\n')
9     except FileNotFoundError:
10        print(f"Error: No se encontro el archivo {ruta_entrada}")
11    except Exception as e:
12        print(f"Error inesperado: {e}")
```

3 Solución 2: Descomposición Funcional (Pipeline)

3.1 Descripción

La solución funcional separa el proceso en cinco tareas independientes: leer el archivo completo, dividirlo en líneas, limpiar cada línea, convertirla a mayúsculas, y escribir el resultado. Cada tarea es una función propia, y una función de orquestación las compone en secuencia.

Listing 2: Tareas independientes del pipeline

```
1 def leer_archivo(ruta_entrada):
2     """Lee y devuelve el contenido completo del archivo de entrada."""
3     with open(ruta_entrada, 'r') as f_in:
4         return f_in.read()
5
6 def dividir_en_lineas(contenido):
7     """Divide el contenido del archivo en una lista de líneas."""
8     return contenido.splitlines()
9
10 def limpiar_linea(linea):
11     """Elimina los espacios en blanco al principio y al final de una línea."""
12     return linea.strip()
13
14 def convertir_mayusculas(linea):
15     """Convierte una línea de texto a mayusculas."""
16     return linea.upper()
17
18 def escribir_archivo(ruta_salida, lineas):
19     """Escribe una lista de líneas en el archivo de salida, una por renglon."""
20     with open(ruta_salida, 'w') as f_out:
21         for linea in lineas:
22             f_out.write(linea + '\n')
```

Listing 3: Orquestacion del pipeline

```
1 def procesar_texto_funcional(ruta_entrada, ruta_salida):
2     """Procesa el archivo aplicando cada transformacion como una tarea separada."""
3     try:
4         contenido = leer_archivo(ruta_entrada)
5         lineas = dividir_en_lineas(contenido)
6
7         lineas_procesadas = []
8         for linea in lineas:
9             linea_limpia = limpiar_linea(linea)
10            linea_mayusculas = convertir_mayusculas(linea_limpia)
11            lineas_procesadas.append(linea_mayusculas)
12
13            escribir_archivo(ruta_salida, lineas_procesadas)
14     except FileNotFoundError:
15         print(f"Error: No se encontro el archivo {ruta_entrada}")
16     except Exception as e:
17         print(f"Error inesperado: {e}")
```

3.2 Explicación de los componentes clave

- **Cinco tareas independientes:** `leer_archivo`, `dividir_en_lineas`, `limpiar_linea`, `convertir_mayusculas` y `escribir_archivo`. Cada una tiene una única responsabilidad y podría probarse o reemplazarse de forma aislada.

- **Lectura completa en memoria:** a diferencia de la versión secuencial, `leer_archivo` carga todo el contenido del archivo en un string antes de procesarlo, y `dividir_en_lineas` genera una lista completa con todas las líneas.
- **Lista intermedia `lineas_procesadas`:** la orquestación acumula el resultado de aplicar `limpiar_linea` y `convertir_mayusculas` a cada línea en una lista antes de escribirla, en lugar de escribir directamente a medida que se procesa.
- `if __name__ == '__main__':` aísla el bloque de ejecución principal (medición de tiempo y rutas de archivo) para que el módulo pueda importarse sin ejecutar el procesamiento automáticamente.

4 Análisis Comparativo

Las dos soluciones se ejecutaron sobre el mismo archivo de entrada (2.100.000 líneas, aproximadamente 60 MB), verificando previamente que ambas produjeran una salida idéntica. Los tiempos promedio obtenidos (varias ejecuciones en el mismo entorno) fueron los siguientes:

Versión	Tiempo promedio (s)	Estructura intermedia	Relativo a secuencial
Secuencial	0.0015	Ninguna (procesa línea por línea)	1.0x
Pipeline (funcional)	0.0020	Lista completa de líneas + lista de resultados	$\approx 1.33x$

Tabla 1: Comparación de tiempos de ejecución entre las dos soluciones.

4.1 Interpretación de los resultados

- La versión **secuencial** es la más rápida porque procesa el archivo línea por línea, sin cargar el contenido completo en memoria y sin funciones propias en el camino crítico: solo llama a los métodos `strip()` y `upper()`, ya optimizados en C.
- La versión con **pipeline funcional se demoró más**, aproximadamente un 33 % más lenta que la secuencial. Esto tiene dos causas que se combinan: (1) cada línea pasa por dos llamadas a función propias (`limpiar_linea` y `convertir_mayusculas`), cada una con el costo fijo de creación de *stack frame* que tiene toda llamada a función en Python; y (2) la solución carga el archivo completo en memoria (`leer_archivo`) y construye dos listas completas (`lineas` y `lineas_procesadas`) en lugar de procesar y escribir de forma incremental. Con millones de líneas, ese trabajo extra de memoria y llamadas se nota, aunque no domina el tiempo total.

5 Conclusiones

1. El pipeline funcional se demoró más que la solución secuencial ($\approx 33\%$ adicional): descomponer el proceso en tareas separadas no trajo una mejora de rendimiento, sino un costo extra. Esto contradice la intuición de que separar responsabilidades siempre optimiza un programa; en Python, cada función añade overhead propio (creación de *stack frame*, resolución de nombres), y ese overhead se multiplica por el número de líneas procesadas.

2. La causa concreta de la lentitud no es solo el número de funciones, sino también *cómo* se usan: `leer_archivo` carga el archivo completo en memoria de una sola vez, y la orquestación construye dos listas completas (`lineas` y `lineas_procesadas`) en vez de procesar y escribir línea por línea como hace la versión secuencial. Ese patrón de "cargar todo primero, procesar después" es el que más peso agrega para archivos grandes.
3. Ese overhead podría reducirse sin perder la separación en tareas independientes: por ejemplo, escribiendo cada línea inmediatamente después de transformarla (en vez de acumularla en `lineas_procesadas`), o leyendo el archivo línea por línea en lugar de cargarlo completo. Esto mantendría la modularidad del pipeline mientras se acerca al comportamiento eficiente de la versión secuencial.
4. A pesar de ser más lento, el pipeline ofrece ventajas que la solución secuencial no tiene: cada tarea (`limpiar_linea`, `convertir_mayusculas`, etc.) puede probarse, reutilizarse o reemplazarse de forma aislada, lo cual facilita el mantenimiento y la extensión del código a futuro.
5. En conclusión, para este caso puntual la solución secuencial es la más conveniente si el único criterio es el tiempo de ejecución. Sin embargo, la decisión entre ambos enfoques no debería basarse solo en el rendimiento: si el código va a crecer, mantenerse o probarse por partes, el costo adicional del pipeline funcional puede estar justificado por la claridad y modularidad que aporta.